

Lab 4: Mapping with LIDAR

CSCI 3302: Introduction to Robotics

Due 03/03/25 @ 11:59pm

Note that this is a 1-week long lab

2 points/day extra credit for early turn-in (up to 10pts)

The goals of this lab are to:

- Implement a basic discrete map representation
- Use coordinate transforms to map sensor readings into the map coordinates
- Understand discrete and algorithmic aspects of mapping

You need:

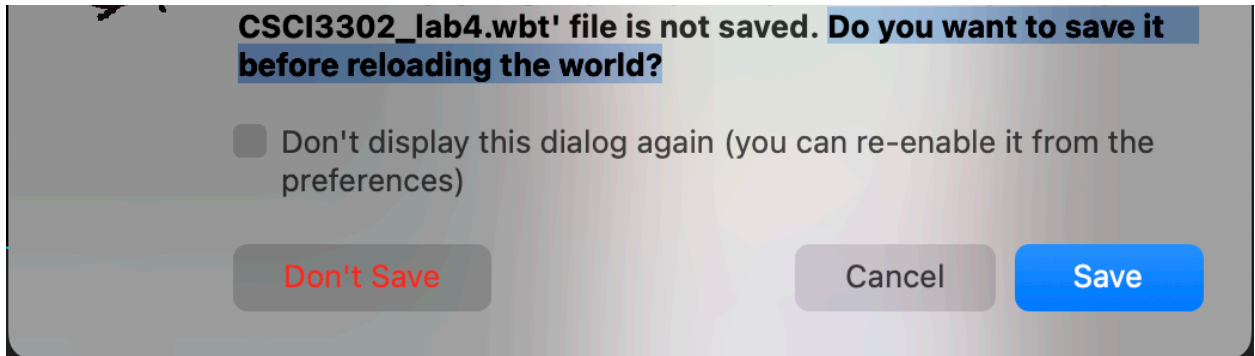
- A working Webots Installation
- The Lab 4 base code from Canvas

Overview

Mapping is a core task in autonomous robotics. Although map representations vary widely in complexity, core challenges persist: uncertain sensing and localization, rapid map corruption, and limited onboard resources for mapping and navigation. In this exercise, use an ePuck robot with a LiDAR sensor to map environmental objects—initially display the robot's visited coordinates, followed by helper functions to (1) convert sensor readings into obstacle locations on the map, (2) store obstacles and visited trajectories in ePuck memory, and (3) visualize them along with free spaces as a map.

Instructions

1. Please do not change the given world file. Also, please click “Don’t Save” before reloading the world.



2. Each group must develop their own software implementation, turned in by one team member with the lab report. Thus, each team should turn in a zip file including:

- The group's code (a .py file)
- A single lab report (a .pdf file)

Before submitting, please verify your code's outputs using our given Webots world environment. If the provided code cannot run in our given world file, a deduction will be assigned.

Part 0A: Preliminaries of Code

Run your code and let the robot travels and ensure:

1. The robot can travel the course and follow the line in clockwise direction for at least two cycles.
2. After each cycle, the terminal prints out "Over the line!" at least once.

If the code cannot make it, understand the code under "Robot Controller" comment, and fix it. Please do not change the world file during this process or anytime working on this lab.

```
#####
# Robot controller #
#####

if state == "line_follower":
    if (gsr[1] < 350 and gsr[0] > 400 and gsr[2] > 400):
        vl = MAX_SPEED*0.3
        vr = MAX_SPEED*0.3
        # Checking for Start Line
        elif (gsr[0] < 310 and gsr[1] < 310 and gsr[2] < 310):
            cnt = 1
            vl = MAX_SPEED*0.3
            vr = MAX_SPEED*0.3
            if cnt > 10:
                cnt = 0
                # Feel free to comment this to make your terminal cleaner
                print("Over the line!")
                # TODO Part 4: Save the map clearly showing the robot's path and the obstacles.
                # Before saving the map,
                # 1. draw ALL the robot's visited pixels again
                # 2. draw ALL the occupied pixels in blue again
                display.imageSave(None, "map.png")

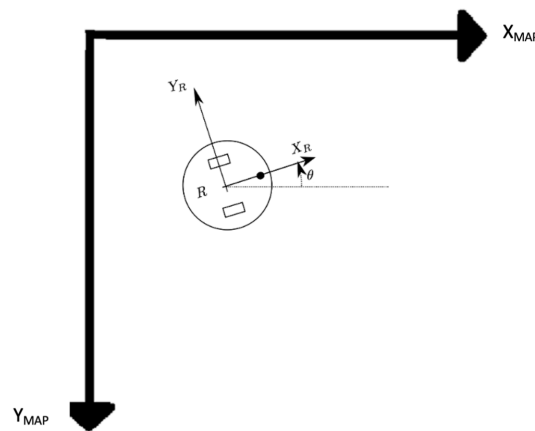
            elif (gsr[2] < 650): # turn right
                vl = 0.2*MAX_SPEED
                vr = -0.85*MAX_SPEED
                cnt = 0
            elif (gsr[0] < 650): # turn left
                vl = -0.85*MAX_SPEED
                vr = 0.2*MAX_SPEED
                cnt = 0
        else:
            # Stationary State
            vl = 0
            vr = 0
    leftMotor.setVelocity(vl)
    rightMotor.setVelocity(vr)
```

Part 0B: Preliminaries on the Transformation Matrix

The map coordinate system has its origin (0,0) at the top-left corner of the arena and its x-axis increases to the right, and its y-axis increases downward. You may refer [Webots Reference Manual on the Display section](#) for more details.

On the other hand, in our defined robot frame, forward motion aligns with the +x-axis, and left corresponds to the +y-axis.

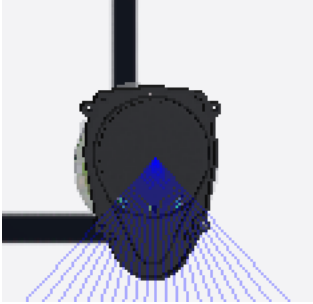
Now, given the defined coordinate systems and the angle θ shown in the figure, find the **transformation matrix** to convert from the robot frame to the map frame. This will also come in handy in Part 3.



Note: Starter code's odometry and initial position are both pre-adjusted for these coordinate systems. **Keep them unchanged.**

Part 1: Preliminaries for using LiDAR

1. Initialize the empty list for the parameter, *lidar_sensor_readings*, which will be used to store LiDAR distance measurements.
2. Precompute the angles of each LiDAR ray in radians and store them in the array *lidar_ray_angles* using the following information:
 - LIDAR_ANGLE_RANGE is the total field of view in radians.
 - NUM_LIDAR_RAYS is the number of rays.
 - Rays are evenly spaced across the total field of view
 - Middle ray points straight ahead at angle 0
 - If you design it properly, *lidar_ray_angles [ray_index]* = angle of that ray in radians



Please also note that:

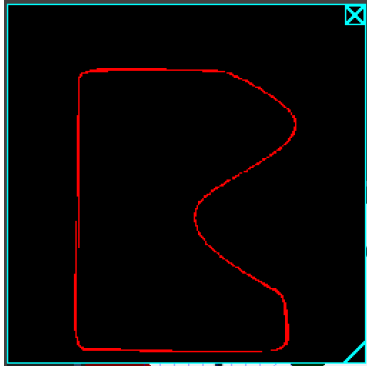
- The sensor object is named *lidar* in the code
- At the beginning of your WHILE loop, we have already added code to get LiDAR sensor readings by `lidar_sensor_readings = lidar.getRangeImage()`, which returns an array of distance measurements based on rays emitted from the lidar sensor.

Part 2: Map Construction

A display node is already attached to the robot's turretSlot, and we want to show the map on this display. **The robot's pose is already in the map frame, but the display's lower resolution requires scaling continuous positions to discrete pixels.** In Part 2, your task is to convert the robot's positions to pixel locations in the display and, as the robot moves, color each visited pixel in red. Please use the following information to complete it.

1. The display has 300 x 300 pixels.
2. In this environment, the robot moves on a 1 m × 1 m floor. Also, both `pose_x` and `pose_y` range approximately from 0 to 1.
3. The following two functions can help us to color a pixel on the display in a selected color:
 1. `display.setColor(int color)`
 2. `display.drawPixel(int x, int y)`
4. Example -- to color the pixel at (0, 0) in red:
 1. `display.setColor(0xFF0000)`
 2. `display.drawPixel(0, 0)`

If you complete the code and color each visited pixel in red, you should see a display similar to the figure after the robot has traveled multiple cycles.

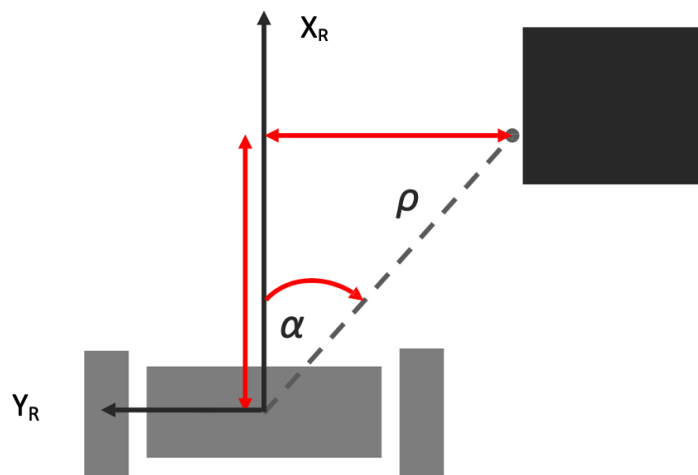


Part 3: Transform LiDAR Readings

Our goal in part 3 is to convert the LiDAR readings into object's positions in the world coordinates.

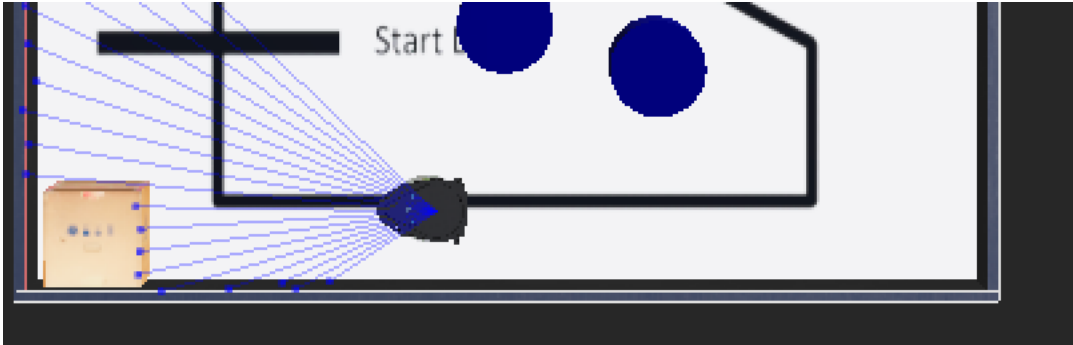
For each LiDAR ray, if the distance measurement is less than the maximum range (`LIDAR_SENSOR_MAX_RANGE` in code):

1. compute the object's poses in the robot's frame (rx and ry).
 - `lidar_sensor_readings` stores the distance measurements of each LiDAR ray (rhos).
 - `lidar_ray_angles` should store the angles of each LiDAR ray (alphas).
 - Read the figure to infer the equations to convert ρ and α into rx , and ry .



2. Apply the homogeneous transformation matrix, inferred in Part 0B, to convert the object's coordinates in the robot's frame (rx , ry) to the coordinates in the map frame (mx , my).

Note: Verify that rx , ry , mx , and my are computed correctly using ray index 10 (angle 0° , the forward ray). For example, when the robot travels along the bottom black line and its forward ray detects the bottom-left object, what is the correct rx value? Also, what are reasonable estimations on mx and my ? Figure these out and ensure your rx , ry , mx , and my provide proper estimates.

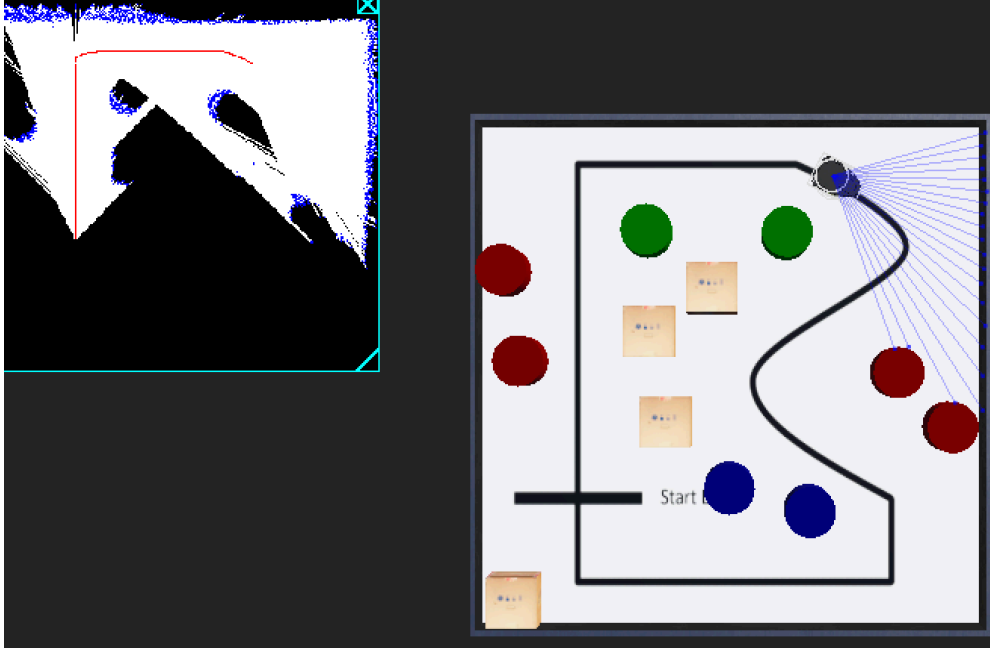


Part 4: Visualize Obstacles onto the Map

In Part 3, you record the locations occupied by obstacles. In Part 4, the goal is to visualize these, along with the robot's visited trajectory and free spaces between the robot and obstacles, on the map. More specifically, every time getting the occupied coordinate:

- Draw the pixel occupied by obstacle in blue.
- Draw the current robot's visited pixel in red.
- Draw a white line between the occupied pixel and the robot's current visited pixel.

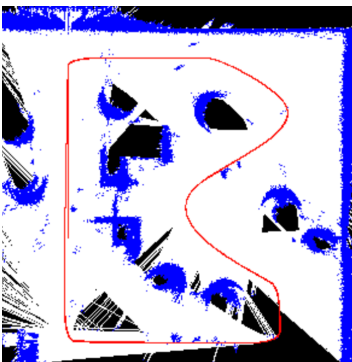
Please note that the drawing order for these three components is critical! You must determine the correct sequence through experimentation so that, when the code runs, the display clearly shows the robot's current position.



Additionally, right before calling `display.imageSave(None, "map.png")`, please do the following steps so that the image clearly shows occupied and visited pixels.

- Draw ALL occupied poses in blue again.
- Draw ALL visited poses in red again.

If you complete all the instructions properly, we should save an image similar to the following one after the robot travels two cycles.



Note:

- Although part 3 gets the map coordinates of the occupied location, you must convert them into display pixel location before drawing them in color.

- The following two functions can help us to color a line on the display in white:
 - `display.setColor(int color)`
 - `display.drawPixel(int start_x, int start_y, int end_x, int end_y)`
- The hexadecimal values of the above colors are shown below:
 - 0xFF0000 - Red - Robot pose
 - 0xFFFFFF - White - Free space
 - 0x0000FF - Blue – Obstacle

Part 5: Lab Report

Create a report that answers each of these questions:

1. What are the names of everyone in your lab group?
2. Roughly how much time did your group spend programming this lab?
3. What is the homogenous transformation matrix you derived? Show how you arrived at your answer to get credit.
4. The display is setup with 300 rows and 300 columns to cover a 1m x 1m area. What is the spatial resolution of each pixel in the map? Show how you arrived at your answer to get credit and show the resolution in the unit m/pixel
5. How would mapping be affected if the odometry is not perfect and has errors?
6. How could you choose a good resolution for your map? Please find the display node in the scene tree in the given world files and explain which parameter of the display node impact our decision in the resolution. Elaborate on what happens if your resolution is too low or too high.
7. Let the agent travels multiple cycles and paste your map.png here. Then:
 - a. Explain what the black color represents in the image.
 - b. Does blue color always indicate obstacles? If no, describe how to reduce blue color in free/open spaces. If yes, explain how you ensure this.